# Urban Expansion and Green Space Loss, Thiruvananthapuram 2000–2025

Full walkthrough of the project: data sources, every processing step, and results.

> **The question:** how much green space did Thiruvananthapuram Corporation lose to buildings between 2000 and 2025? Where was it lost, why there, and which wards need attention first?
>
> **How it is built:** 7 Python scripts in `scripts/`, run in order. Each step reads the previous step's files from `data/processed/`.
>
> **Map projection:** UTM zone 43N (EPSG:32643). It works in metres, so areas and distances are correct.
>
> **Land-cover classes:** 1 Built-up, 2 Tree cover, 3 Other vegetation, 4 Water, 5 Bare/sand. "Green" means classes 2 and 3 together.

| Step | Script | What it does |
|---|---|---|
| 1 | `step1_base_data.py` | Wards, corporation, study area, OSM roads/buildings/land use |
| 2 | `step2_composites.py` | Cloud-free satellite composites and ward population (Earth Engine) |
| 3 | `step3_classification.py` | Random Forest land-cover maps |
| 4 | `step4_change.py` | Consistent time series, change detection, ward statistics |
| 5a | `step5_sedona.py` | Big data processing with Apache Sedona on Spark |
| 5b | `step5_postgis.py` + `sql/step5_postgis.sql` | Spatial database analysis in PostGIS |
| 6 | `step6_analytics.py` | Hotspots, association rules, case studies, ward priority index |
| 7 | `step7_prepare.py` + `dashboard/app.py` | Interactive Streamlit dashboard |

## Step 1: Study area and base data

### Data sources

- **Ward boundaries:** K-SMART 2024 ward map from GitHub (madpk/kerala-ward-map), file `Kerala_ward_boundaries.geojson`. It covers all of Kerala.
- **OpenStreetMap:** the Geofabrik "southern-zone" extract, `southern-zone-latest.osm.pbf`. Geofabrik was used because the Overpass API was down.

### What was done

1. Kept only Thiruvananthapuram Corporation (`lb_code = C010100`), which gives **101 wards**.
2. Deleted the surveyor names and phone numbers (personal data). Renamed the columns to `ward_no`, `ward_name` and `ward_name_ml`.
3. Converted the wards to UTM, repaired invalid shapes and calculated each ward's area in km².
4. **Corporation outline:** merged all wards into one shape, buffered out 10 m and back in 10 m to close thin gaps between wards. The corporation is about 213 km².
5. **Study area:** the corporation plus a 2 km buffer, so growth at the edge is not cut off. About 404 km², including some sea.
6. Read three layers from the OSM file, limited to the study area:

- **Roads:** about 29,000 segments, grouped into highway, major, minor and local.
- **Buildings:** about 203,000 footprints.
- **Land-use and natural polygons.**

### Outputs

- `base.gpkg` with six layers: wards, corporation, study_area, roads, buildings, landuse.
- `corporation_boundary.geojson` and `study_area.geojson`, used as the upload area for Google Earth Engine.
- Map `outputs/maps/step1_base_map.png`.

## Step 2: Satellite composites and population

**Data source:** Google Earth Engine, cloud project `geometric-wall-477105-i2`.

**Method:** for each year, all clear images from the **dry season (November to April)** were combined into a **median composite**, one cloud-free image.

### Landsat at 30 m (Collection 2, Level 2)

- Clouds removed with the `QA_PIXEL` bits (dilated cloud, cirrus, cloud, shadow) and the `QA_RADSAT` saturation band.
- Reflectance scaled with ×0.0000275 − 0.2. Land surface temperature (LST) calculated from the thermal band, in °C.

| Year | Sensors | Scenes | Dry seasons used |
|---|---|---|---|
| 2000 | Landsat 5 + Landsat 7 | 14 | Nov 1999 – Apr 2001 (two seasons; Landsat 5 alone had 0 scenes) |
| 2010 | Landsat 5 + Landsat 7 | 21 | two seasons |
| 2017 | Landsat 8 | 22 | one |
| 2025 | Landsat 8 + Landsat 9 | 42 | one |

### Sentinel-2 at 10 m (L2A)

- Clouds removed with **Cloud Score+** (`cs_cdf ≥ 0.60`).
- The 20 m SWIR bands were resampled to 10 m with bilinear smoothing.
- 2019 used 58 scenes and 2025 used 80. 2017 had only 3 scenes, so it was dropped.

### Added to every composite

- **NDVI** (vegetation), **NDBI** (built-up) and **MNDWI** (water).
- A `clear_obs` band: how many clear views each pixel had.

**Download:** each composite was saved as a GeoTIFF on the UTM grid with the `geedim` library. Landsat images are 754×1039 pixels; Sentinel-2 images are 2260×3114.

### Population (WorldPop, 100 m grid)

Population was summed per ward from two releases:

- Old release: 2000 = 920k, 2010 = 947k, 2020 = 952k.
- New release (R2025A): 2017 = 1.149M, 2025 = 1.192M.

> The two WorldPop releases use different models. Never compare a year from one release with a year from the other.

**Outputs:** `composites/*.tif`, `composites_summary.csv`, `ward_population.csv`, `step2_preview.png`.

## Step 3: Random Forest land-cover classification

### 3a. Reference products from Earth Engine

Downloaded as `reference/ref_30m.tif` and `ref_10m.tif`:

- ESA WorldCover 2021
- Dynamic World, most common dry-season label for 2017, 2019 and 2025
- GHSL built-up surface for 2000, 2010, 2015, 2020 and 2025

> **Problem:** WorldCover and Dynamic World disagree badly in this city. Dynamic World calls about 80% of it built-up. WorldCover calls about 60% of it trees, because houses here sit under coconut canopy. Neither product could be used alone as training labels.

### 3b. Training labels for Sentinel-2 (2019, 2025)

A pixel became a training label only if it passed all four checks:

1. WorldCover and Dynamic World agree on its class.
2. At least 7 of the 9 pixels in its 3×3 window have the same label (no mixed edges).
3. GHSL backs it up: built-up needs at least 20% built surface; other classes need at most 10%.
4. Its spectra fit the class. For example, trees need NDVI > 0.55 and water needs MNDWI > 0.

### 3c. Training labels for Landsat (2000, 2010, 2017, 2025)

These come from the **Sentinel-2 2025 map**, not from the reference products. Trained on the reference products alone, Landsat found only the dense core (about 10% built-up).

1. Each 30 m cell takes the 10 m class that covers at least half of it.
2. Built-up must have GHSL ≥ 20% in that year.
3. For years before 2025, only pixels whose spectra barely changed since 2025 are kept (change magnitude below the 75th percentile for their class).

### 3d. The model

- **Features (9):** blue, green, red, NIR, SWIR1, SWIR2, NDVI, NDBI, MNDWI.
- **LST was left out** because it is missing over the north-west ~20% of the study area in every Landsat year.
- **Samples:** up to 2,000 per class.
- **Test set:** the area was cut into 1 km blocks and 30% of the blocks were held out. This spatial split stops nearby pixels leaking between training and testing.
- **Random Forest:** 200 trees, `max_features = sqrt`, seed 42.
- **Final maps:** the model was refit on all samples and every pixel was predicted. A 3×3 majority filter removed speckle.

### Accuracy on the held-out blocks

| Map | Overall accuracy | Kappa |
|---|---|---|
| Sentinel-2 2019 | 0.959 | 0.948 |
| Sentinel-2 2025 | 0.958 | 0.947 |
| Landsat 2000 | 0.709 | 0.635 |
| Landsat 2010 | 0.757 | 0.696 |
| Landsat 2017 | 0.823 | 0.778 |
| Landsat 2025 | 0.819 | 0.773 |

NDVI was the most important feature.

### Outputs

- `landcover/lc_*.tif`: the six land-cover maps
- `training_samples.csv`
- Accuracy, confusion-matrix and feature-importance CSVs
- `landcover_area.csv`
- `validation_points.gpkg`: 50 points per class per 2025 map, for checking by eye
- Map `step3_landcover.png`

> **Problem found:** the separate yearly maps showed almost no trend (built-up 20.9% in 2000, 23.0% in 2025). Landsat 2000 also shows 6.7% bare/sand, which is suspicious. Step 4 fixes the trend problem.

## Step 4: Change detection

### 4.1 Backdating

The series starts from the 2025 map and steps back to 2017, then 2010, then 2000. Rules at each step back:

- The change-vector magnitude is measured between the year and the next later year, using the 6 bands plus NDVI, each standardised.
- A pixel counts as "changed" if its magnitude is greater than the mean plus k × the standard deviation. Changed patches smaller than 3 pixels are ignored.
- An unchanged pixel keeps the later year's class. A changed pixel takes that year's own Random Forest class.
- **A pixel cannot be built-up in an earlier year unless it is also built-up in the later year.**

**Choosing k:** k was tested at 0, 0.25, 0.5, 0.75, 1.0 and 1.5. The value whose built-up trend best matched GHSL was **k = 0.25**. The sweep is saved in `backdating_k_sensitivity.csv`.

### Final consistent series (corporation)

| Year | Built-up | Green | GHSL built surface |
|---|---|---|---|
| 2000 | 15.6% | 77.7% | 13.2% |
| 2010 | 18.7% | 75.9% | 18.2% |
| 2017 | 20.6% | 74.9% | 20.8% |
| 2025 | 23.0% | 74.2% | 21.3% |

### 4.2 Transitions inside the corporation, 2000–2025

- Green fell from **165.15 km² to 157.72 km²**, a net loss of **7.43 km² (−4.5%)**.
- **12.17 km² of green became built-up.**
- Built-up grew by 15.6 km².
- 5.91 km² became green.
- Green was lost at about 0.53 km² per year.

Green that became built-up, by period: 2000–2010 = 5.26 km², 2010–2017 = 2.94 km², 2017–2025 = 3.43 km².

The 2000→2025 change raster has 7 codes; code 3 means green → built-up.

### 4.3 Ward statistics (all 101 wards)

- Built-up, green and tree share for every year, plus the 2025 share at 10 m from Sentinel-2.

- Change in percentage points.
- Green space per person ($m^2$).

| Largest green loss, 2000–2025 | Change |
|---|---|
| Kamaleswaram | −29.6 points |
| Sreevaraham | −26.1 points |
| Kalippankulam | −25.9 points |
| Pettah | −18.0 points |
| Kesavadasapuram | −14.3 points |

**Outputs:** `landcover_consistent/*.tif`, `change_2000_2025.tif`, `transitions.csv`, `change_summary.csv`, `ward_change.csv`, `change.gpkg`, and 3 maps or charts.

## Step 5a: Big data with Apache Sedona on Spark

**Idea:** turn every pixel into a point, so the analysis becomes a classic big spatial data job.

### Export to Parquet (`data/processed/bigdata/`)

- **4.04 million** Sentinel-2 points
- **451,000** Landsat points, each with its land cover for the four years and its change code
- Wards, 203,000 buildings and 29,000 roads

**Setup:** Spark in local mode with Sedona 1.8.0. The Sedona jar files are put directly on the classpath. The empty `tools/hadoop` folder is only there so Spark starts on Windows.

### 5.1 Benchmark: points-in-ward join, then count per ward (4.04 million points)

| Method | Time |
|---|---|
| Spark with no spatial optimisation (nested loop) | **192 s** |
| Sedona broadcast index join (R-tree) | **3.0 s** |
| Sedona Quadtree partitioned join | 17.6 s |
| Sedona KDB-tree partitioned join | 34.0 s |
| GeoPandas on one machine | **1.6 s** |

> **Conclusion:** at this data size a single machine wins. Sedona's value is that it scales, not that it is faster here. The query plans were saved to `outputs/logs/step5_plans.txt`, read as MapReduce: map, combine, shuffle, reduce.

### 5.2 Partitioning the 4.04 million points

| Scheme | Partitions | Balance | Area each partition covers |
|---|---|---|---|
| Hash (Spark default) | 24 | Perfect | 99.9% of the whole area (no spatial locality) |
| KDB-tree | 33 | Uneven (CV 1.79) | 2% |
| Quadtree | 49 | CV 0.44 | 1.3% |

### 5.3 Analytics

- **Ward land-cover shares** match Step 4 within 0.53 percentage points.
- **Buildings and road length per ward.**

- **Distance to the nearest main road** for every pixel that was green in 2000 (kNN join). Expansion follows roads:

| Distance to main road | Green that became built-up |
|---|---|
| 0–100 m | 12.4% |
| 100–250 m | 8.3% |
| 250–500 m | 6.6% |
| 500–1000 m | 3.8% |
| 1–2 km | 2.3% |

- **H3 hexagon grid** (resolution 9, about 0.1 km$^2$ each): **2,082 hexagons**, each with its green → built-up figures. Saved in `sedona.gpkg`.

## Step 5b: PostGIS

Run on local PostgreSQL 13 with PostGIS.

### Loading (database `tvm_green`, schema `tvm`)

- Vector layers loaded with `to_postgis`.
- The 451,000 pixels loaded with `COPY`.
- The change raster loaded with `raster2pgsql` in 64×64 tiles.

### SQL steps

1. GiST spatial indexes, plus a partial index on main roads only.
2. Each pixel's ward stored as a column.
3. A geohash-5 column, then `CLUSTER` to store neighbouring points together on disk.
4. A **LIST-partitioned table** with one partition per geohash cell: **30 partitions**.
5. Views and materialised views: ward green loss calculated two ways (from the pixel points and from the raster); buildings and roads per ward; kNN nearest main road (`<->` operator); road-distance bands; conversion by OSM land use; corporation totals.

### Benchmark (with GiST index vs. without)

| Query | With index | Without index |
|---|---|---|
| Pixels in wards | 0.53 s | 4.29 s |
| Buildings in wards | 0.87 s | 20.6 s |
| Range query, 1 km around Technopark | 0.009 s | 0.073 s |
| kNN nearest main road | 0.13 s | 14.5 s |
| Partition pruning | 0.010 s (1 of 30 partitions read) | 0.076 s (all 30 read) |

**Cross-check:** PostGIS gives the same answers as Sedona and Step 4: **12.17 km²** of green → built-up and the same road-distance bands.

## Step 6: Analytics and decision support

### 6.0 Extra layers from Earth Engine (resampled onto the 30 m grid)

- **Copernicus GLO-30 DEM:** elevation, and slope calculated from it.
- **MERIT Hydro HAND** (height above nearest drainage). Land with HAND ≤ 5 m counts as low-lying.

- **Sentinel-1 flood map, August 2018.** Water = radar backscatter below −16 dB that dropped more than 3 dB against the Feb–Apr 2018 median. Permanent water and slopes of 5° or more removed.
- **MODIS LST at 1 km**, dry seasons 2002/03 and 2024/25. MODIS has no gaps, unlike Landsat LST.

### 6.1 Hotspots

**Getis-Ord Gi*** on the hexagons (6 nearest neighbours, 999 permutations):

- **262** significant hot hexagons ($p \leq 0.05$), **115** after FDR correction.
- They contain 4.55 km² of the green → built-up conversion.

**Moran's I** (spatial clustering): 0.485 for hexagons, 0.343 for wards, both $p = 0.001$.

**DBSCAN** on the converted pixels (radius 100 m, at least 12 pixels):

- **250 clusters**, holding 57.5% of all converted pixels.
- Largest: Sreevaraham 0.44 km², Kalippankulam 0.40 km², Chakkai 0.28 km².

### 6.2 Association rules (FP-Growth, Spark MLlib)

- **182,192 transactions.** Each is one pixel that was green in 2000.
- **Items** describe where the pixel is: distance to a road, to the city centre and to Technopark; elevation; slope; low-lying or not; flooded in 2018; built-up around it in 2000; ward density; OSM land use; tree or other vegetation.
- **Outcome item:** "→ built-up" or "stayed green".
- **Thresholds:** minimum support 0.003, minimum confidence 0.10.

Results:

- City-wide conversion rate: **7.42%**.
- 112,441 frequent itemsets found.
- After removing redundant rules: **146 conversion rules** and **17 stayed-green rules**.
- **Strongest rule:** high built-up around it in 2000 + within 3 km of the centre + flat → **39% converted, lift 5.3** (5.3 times the city-wide rate).

### 6.3 Case studies

**Green space per person** (compared only within one WorldPop release):

- Old release: 179.5 m² in 2000 → 170.6 m² in 2010.
- New release: 138.7 m² in 2017 → 132.3 m² in 2025.
- Wards with less than 50 m² per person rose from 23 to 27 between 2017 and 2025.

**Heat (Landsat LST):**

- Green that became built-up warmed by **+3.04 °C**; green that stayed green warmed by +1.45 °C. That is **1.59 °C extra warming**.
- Only 76% of the corporation has LST in both years.
- LST in 2025 by class: built-up 38.7 °C, trees 34.6 °C, water 33.1 °C.

**Heat (MODIS, ward level):**

- Ward green share vs. temperature: Spearman $\rho = -0.77$. Greener wards are cooler.
- Ward green change vs. temperature change: $\rho = -0.47$.

**Floods:**

- There are 55 km² of low-lying land.
- Green on low-lying land became built-up at 7.8%, against 7.2% on higher ground.
- The 2018 radar flood map shows only **0.32 km²**, too small to use. Floods are shown through HAND low-lying land instead.

### 6.4 Ward priority index (decision support)

**Seven indicators**, each scaled 0–1 after clipping the top and bottom 2%:

- Green loss 2000–2025
- Green loss 2017–2025
- Share of the ward inside hotspot hexagons
- Share of low-lying green converted
- Temperature (LST)
- Green per person (less green = higher priority)
- Population density

**Weighting schemes:** the main index uses equal weights. It was checked against 3 other schemes (loss pressure, climate risk, people). The ranking stays stable: Spearman ρ ≥ 0.967, and 16–19 of the top 20 wards stay the same.

| Rank | Ward | Score |
|---|---|---|
| 1 | Sreevaraham | 94.8 |
| 2 | Perunthanni | 87.6 |
| 3 | Kamaleswaram | 86.1 |
| 4 | Pettah | 76.6 |
| 5 | Kalippankulam | 74.3 |

**Outputs:** `analytics.gpkg` (hex_hotspots, dbscan_clusters, ward_priority), the `step6_*.csv` files and 4 maps.

## Step 7: Dashboard

**Preparation:** every result is pre-computed into small files in `dashboard/data/`, so the app needs neither Spark nor Earth Engine:

- `wards.geojson`: all ward indicators
- `hexes.geojson`, `clusters.geojson`, `roads_main.geojson`, `corporation.geojson`
- PNG map overlays, converted to the web map projection
- `px30.parquet`
- Tables in `tables/*.csv`

**The app:** Streamlit, 7 pages:

1. Overview
2. Map explorer
3. Ward drill-down
4. Hotspots & rules
5. Spatial query: draw a shape or a buffer and get the numbers inside it. Uses PostGIS when available, otherwise the local Parquet file.
6. Priority (decision support)
7. Big data engine

**Settings:** `.streamlit/config.toml` makes the app open only on this computer (localhost) and sets the green theme. The base map is Esri.WorldGrayCanvas.

**Other deliverables:** the 19-slide deck in `report/`; the written report in `final report/` as .docx, .pdf and .html.

## Limits to state in the presentation

- The Landsat accuracy (0.71–0.82) is measured against automatically made labels, not against checks done by eye. The validation points exist, but they only give an independent accuracy once they are labelled.
- The 6.7% bare/sand in Landsat 2000 is suspicious. It causes the apparent green gain in the north-west coastal wards (Pallithura, Powndukadavu).
- Report total green only. Trees and other vegetation can't be reliably separated at 30 m.
- The green-per-person change from 2010 to 2017 is not valid, because the WorldPop release changes in between.
- No deep learning model was trained. Dynamic World and Cloud Score+ are only pre-trained inputs.